

Machine Learning Assignment 2

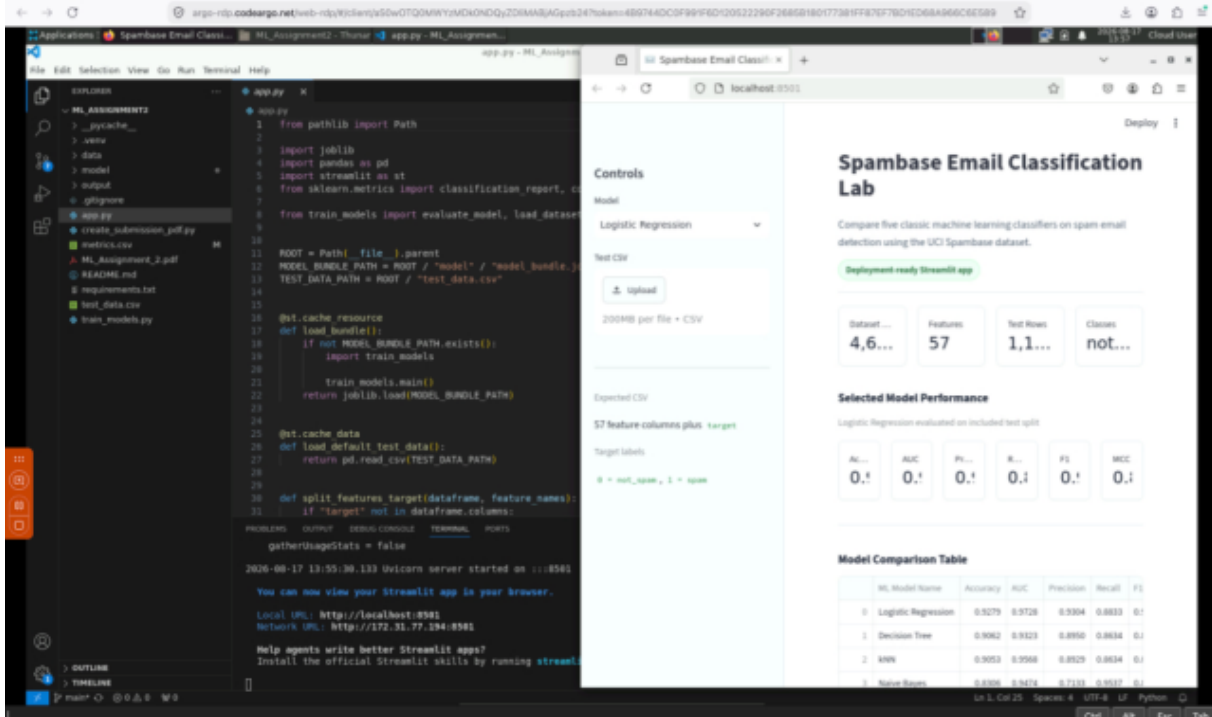
Spam Email Classification Using Machine Learning

This PDF contains the required GitHub repository link, live Streamlit app link, BITS Virtual Lab execution screenshot, README content, model comparison table, model observations, and final submission checklist.

GitHub Repository Link: https://github.com/MuthukumarM16/ML_Assignment2

Live Streamlit App Link: <https://mlassignment2-xpdyarvzmwdeevknpesn6.streamlit.app/>

BITS Virtual Lab Screenshot



a. Problem Statement

The objective of this project is to build and compare multiple supervised machine learning classification models for identifying whether an email is spam or not spam. Each record represents an email summarized through word-frequency, character-frequency, and capital-letter sequence features. The project includes model training, evaluation using standard classification metrics, and an interactive Streamlit web application where test data can be uploaded and model performance can be viewed.

b. Dataset Description

This project uses the Spambase dataset, a public classification dataset from the UCI Machine Learning Repository.

Property	Value
Number of instances	4,601
Number of input features	57
Target classes	Not spam and spam

Property	Value
Problem type	Binary classification
Feature type	Numeric word-frequency, character-frequency, and capital-run-length attributes
Target meaning	0 means not spam and 1 means spam

The dataset satisfies the assignment constraints because it has more than 500 instances and more than 12 features.

c. GitHub Repository Link

GitHub Repository Link: https://github.com/MuthukumarM16/ML_Assignment2

Live Streamlit App Link: <https://mlassignment2-xpdyarvzmwdeevknnpesn6.streamlit.app/>

d. Models Used

The following models were trained and evaluated on the same stratified test split: Logistic Regression, Decision Tree Classifier, K-Nearest Neighbor Classifier, Gaussian Naive Bayes Classifier, and Random Forest Classifier.

Model Comparison Table

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9279	0.9728	0.9304	0.8833	0.9062	0.8485
Decision Tree	0.9062	0.9323	0.8950	0.8634	0.8789	0.8027
kNN	0.9053	0.9568	0.8929	0.8634	0.8779	0.8009
Naive Bayes	0.8306	0.9474	0.7133	0.9537	0.8162	0.6893
Random Forest (Ensemble)	0.9322	0.9826	0.9332	0.8921	0.9122	0.8576

Model Performance Observations

ML Model Name	Observation about model performance
Logistic Regression	Performed strongly with high accuracy, AUC, precision, F1 score, and MCC. Many spam patterns can be separated well using a linear decision boundary after feature scaling.
Decision Tree	Gave good performance but was weaker than Logistic Regression and Random Forest. A single tree is sensitive to the train-test split and individual feature thresholds.
kNN	Achieved performance close to the Decision Tree. Distance-based classification worked reasonably well after scaling but did not outperform the stronger models.
Naive Bayes	Achieved the highest recall, meaning it detected most spam emails, but lower precision means it also marked more non-spam emails as spam.
Random Forest (Ensemble)	Achieved the best overall performance with the highest accuracy, AUC, F1 score, and MCC. The ensemble handled non-linear feature interactions well.
Overall Winner for this dataset	Random Forest is the overall winner because it achieved the best balance across all required metrics.

Streamlit Application Features

The Streamlit application includes CSV test data upload, model selection dropdown, evaluation metrics display, classification report, confusion matrix, test data preview, and an overall model comparison table.

Repository Structure

The repository contains app.py, train_models.py, requirements.txt, README.md, test_data.csv, metrics.csv, data/spambase.data, data/spambase.names, data/spambase.csv, and saved model artifacts under the model folder.

Final Submission Checklist

Checklist Item	Status
GitHub repo link works	Done
Streamlit app link opens correctly	Done
App loads without errors	Done
All required Streamlit features implemented	Done
README.md updated	Done
README content added in submitted PDF	Done
BITS Virtual Lab screenshot included	Done